

Federated QoS Offload Decisions for OneM2M IoT Middleware

Project Report

MSc Research Project / (Research) Practicum / Internship
Cloud Computing

Uday
Student ID: TBD

School of Computing
National College of Ireland

Supervisor: TBD

September 2026

Federated QoS Offload Decisions for OneM2M IoT Middleware

Uday
TBD

Abstract

Et-Tousy, Zyane & Sharif (2026), “Adaptive QoS Management in OneM2M Standard: Machine Learning and Deep Learning for IoT Network Optimization” (Journal of Network and Systems Management), trains a centralized Random Forest on pooled QoS telemetry (RTT, CPU, RAM, success rate) to decide whether IoT traffic should stay local, partially offload, or fully offload to the cloud, reaching 98% accuracy. The paper explicitly names a gap it does not resolve: its centralized learning mechanism “might not scale well or guarantee data privacy in distributed IoT environments,” with federated learning proposed only as future work. This project reproduces the centralized baseline on synthetic QoS telemetry from 6 non-IID simulated IoT edge sites (each dominated by one of the paper’s own uniform/burst/real-time traffic scenarios), then builds and evaluates the federated alternative the paper names but does not build: one Random Forest per site, trained on local data only, combined at inference time by averaging predicted probabilities, so no raw telemetry ever leaves its site. Across 5 seeds, the federated ensemble reaches 99.5% accuracy and 99.5% macro-F1 versus the centralized baseline’s 99.9%/99.96% – a small utility cost for removing the need to centralize raw data – while a single site’s own local model used in isolation, with no federation at all, reaches only 97.6% accuracy and 97.4% macro-F1. Federation recovers almost all of the gap between going it alone and pooling everything centrally, without requiring the centralization the baseline paper’s own limitations section flags as a privacy risk.

1 Introduction

IoT middleware platforms built on standards like OneM2M need to decide, in real time, whether to process a device’s traffic locally, offload part of it to the cloud, or fully migrate processing to the cloud, based on current network and resource conditions. Static, manually-configured rules cannot keep up with dynamic, bursty IoT traffic, motivating machine-learning-driven QoS management.

Et-Tousy, Zyane & Sharif Et-Tousy et al. (2026) address exactly this with a centralized Random Forest classifier trained on pooled QoS telemetry (round-trip time, CPU, RAM, success rate) from all connected devices, achieving 98% accuracy at recommending Keep-Local / Partial-Offload / Full-Offload decisions. Their own Discussion section names a limitation they do not resolve: centralizing telemetry from every device “might not scale well or guarantee data privacy in distributed IoT environments,” proposing federated learning only as future work.

Research question: can a federated learning alternative – training one model per IoT site on local data only, and combining them without ever pooling raw telemetry – recover most of the accuracy of the centralized baseline, while removing the need to centralize data at all?

Objectives:

- Reproduce the baseline paper’s centralized Random Forest classifier and its exact decision thresholds and class-imbalance handling (SMOTE), on synthetic telemetry from simulated, non-IID IoT sites.
- Build the federated alternative the paper names as future work but does not build: per-site local Random Forests combined at inference time.
- Quantify what a site loses by going it alone (no federation, no centralization) as a lower-bound comparison, to establish whether federation is actually worth building relative to that baseline.
- Evaluate all three variants on the same held-out, pooled test set across multiple random seeds.

The remainder of this report is structured as follows: Section 2 reviews the baseline paper and the specific gap it names; Section 4 describes the reproduced baseline and the federated design; Section 5 describes the synthetic multi-site telemetry generator, training pipeline, and deployment; Section 6 reports results across 5 seeds; Section 7 discusses limitations and future work.

2 Related Work

IoT middleware standards such as OneM2M provide interoperability across vendors and device types, but leave dynamic Quality-of-Service (QoS) management and traffic-offload decisions largely unaddressed, forcing operators to rely on manual, reactive intervention when systems become overloaded. This literature review first contextualizes the baseline paper’s contributions in Section 2.1, identifies the specific gap this project targets in Section 2.2, and then surveys five bodies of literature that motivate and contextualize the federated learning alternative: federated learning for IoT and edge computing (Section 2.3), QoS management and offloading in IoT (Section 2.4), privacy-preserving distributed learning (Section 2.5), non-IID data challenges in federated learning (Section 2.6), class imbalance and ensemble methods (Section 2.7), and autonomic computing with MAPE-K (Section 2.8).

2.1 Baseline: ML/DL for OneM2M QoS Optimization

The baseline for this project is Et-Tousy, Zyane & Sharif Et-Tousy et al. (2026), “Adaptive QoS Management in OneM2M Standard: Machine Learning and Deep Learning for IoT Network Optimization” (Journal of Network and Systems Management, May 2026, DOI: 10.1007/s10922-026-10071-4). The paper collects real-time QoS telemetry (RTT, CPU, RAM, success rate) from Azure IoT devices via a MAPE-K monitoring loop, labels each window as Preferable / Acceptable / Critical using fixed thresholds, and trains classical ML models (Random Forest, SVC, KNN, XGBoost, LightGBM), ensembles (Voting/Stacking of RF+LightGBM), and deep learning models (CNN, LSTM, VAE+RF) to

recommend a traffic-offload decision: keep processing local, partially offload to the cloud, or fully offload. Their best model (a Random Forest / LightGBM ensemble) reaches 98% accuracy and is deployed as a live API that drives the Analyzer stage of a MAPE-K loop, reducing RTT by 30-50% and increasing success rates by 10-15% across uniform, burst, and real-time traffic scenarios.

The same authors' 2025 conference paper Et-Tousy et al. (2025) applies machine learning to OneM2M QoS optimization with SMOTE-based balancing and ensemble methods (Voting/Stacking) for traffic classification, achieving 98% F1-score. A companion paper Zyane and Et-Tousy (2025) explores deep learning (AutoEncoders, LSTM, CNN) for OneM2M QoS parameter optimization.

2.2 The Gap This Project Targets

The paper is explicit about what it does not address. In their own Discussion section: "the system currently relies on a centralized learning mechanism, which might not scale well or guarantee data privacy in distributed IoT environments. To overcome this, we plan to explore federated learning." This is stated as future work, not built or evaluated in the paper itself. This project reproduces that centralized baseline, then builds and evaluates the federated alternative the baseline paper names but does not implement.

2.3 Federated Learning for IoT and Edge Computing

Federated learning is a well-established response to exactly the centralization concern the baseline paper names – training models across distributed clients without centralizing raw data. Trigka and Dritsas Trigka and Dritsas (2025) survey federated learning for IoT, covering techniques, privacy, energy efficiency, non-IID data challenges, and scalability across IoT applications (smart cities, healthcare, industrial automation). Wu et al. Wu et al. (2024) provide a comprehensive survey of topology-aware federated learning in edge computing, reviewing star, hierarchical, peer-to-peer, and mesh topologies and their implications for latency, communication overhead, and fault tolerance in 5G/6G edge networks. Li Li (2024) surveys data security and privacy-preserving techniques (secure multi-party computation, homomorphic encryption, secret sharing, differential privacy) for federated learning in edge-IoT systems.

Abreha et al. Abreha et al. (2026) provide the most comprehensive recent survey, covering federated learning for edge computing-enabled Artificial Intelligence of Things (AIoT), emphasizing synergies between FL and edge computing for low-latency, privacy-preserving distributed learning. A 2025 IEEE Communications Surveys & Tutorials paper Minwuye et al. (2025) examines the three-way trade-off between privacy, model performance, and network efficiency in IoT-deployed federated learning, evaluating Privacy-Preserving FL (PPFL) solutions across diverse QoS and network constraints.

Xiao et al. Xiao et al. (2024) propose a federated deep Q-network (DQN) approach for task offloading in mobile edge computing-enabled heterogeneous networks, where each user device optimizes both the offloading decision (for processing) and the participation decision (for learning), minimizing total energy consumption under deadline constraints while accounting for the cost of the federated learning process itself – demonstrating that federating the learning process carries measurable overhead that must be explicitly optimized. Wu et al. Olvera-Córdova et al. (2024) apply distributed deep reinforcement learning (Actor-Critic) for dynamic task offloading in 5G edge-cloud continuum, maxi-

mizing Quality-of-Experience (battery level) subject to latency requirements – confirming that offloading decisions must adapt to the environment’s evolution.

2.4 QoS Management and Offloading in IoT

2.4.1 OneM2M and IoT Middleware

The OneM2M standard provides interoperability but lacks built-in mechanisms for dynamic QoS optimization and overload management, motivating the baseline paper’s contribution. Recent work on OneM2M addresses this gap across several dimensions. Vo et al. Vo et al. (2023) propose an enhanced interoperating mechanism between OneM2M and Open Connectivity Foundation (OCF) using a rules engine and interworking proxy, addressing heterogeneous IoT network integration. Kim et al. Kim et al. (2023) design a blockchain-based interworking proxy between OneM2M and LWM2M to ensure trusted, auditable cross-standard communication with data integrity guarantees. B et al. B et al. (2024) present a lightweight OneM2M implementation for resource-constrained IoT devices, addressing protocol interoperability and latency challenges. Ihita et al. Ihita et al. (2023) evaluate security mechanisms for OneM2M-based smart city networks using the OM2M open-source implementation.

Et-Tousy and Zyane Et-Tousy and Zyane (2025) analyze QoS in OneM2M across HTTP, MQTT, and CoAP protocols with automatic traffic SLA management. Extensive protocol comparison shows CoAP exhibits best efficiency (low RTT, minimal resource usage), MQTT QoS Level 2 ensures highest reliability for mission-critical applications, and HTTP suffers from excessive overhead limiting scalability. Queueing theory modeling categorizes system behavior into preferable/acceptable/critical zones based on utilization, validated through real-world overload scenarios. Automated MAPE-K-based traffic orchestration prevents overload by dynamically prioritizing data streams and offloading excess traffic to cloud resources.

2.4.2 Edge Computing and Offloading Decisions

Deep reinforcement learning has emerged as a dominant approach for dynamic task offloading. Zhang et al. Zhang et al. (2024) propose a hybrid DRL-based (SD3: Softmax Deep Double Deterministic Policy Gradients) task offloading scheme for D2D-assisted cloud-edge-device collaborative networks, jointly optimizing offloading decisions, transmission power, transmission rate, and computational resource allocation to minimize time and energy consumption. Li et al. Li et al. (2024) address mobility-aware partial task offloading with DRL, introducing virtual energy queues and Lyapunov optimization to handle long-term energy constraints under stochastic user movement and time-varying channel conditions.

Pang et al. Pang et al. (2025) propose DAPO (Dynamic Adaptive Partitioning and Offloading), a DDPG-based framework for multi-user, multi-edge server collaborative LLM inference, jointly optimizing model partition points and offloading destinations while accounting for user mobility – achieving 27% lower latency and 18% lower energy consumption than PPO and A2C. Qu et al. Qu et al. (2025) propose KroQoSNet, a lightweight Kronecker-enhanced CNN combined with a deep Q-network (DQN) scheduler for protocol-agnostic, encrypted traffic-aware QoS prediction and adaptive protocol selection, demonstrating real-time inference on Raspberry Pi 4 with over 65.8% model compression.

2.5 Privacy-Preserving Distributed Learning

Privacy preservation in distributed learning spans cryptographic and perturbation-based approaches. Xu et al. (2025) present Sequoia, a framework for privacy-preserving machine learning (PPML) over distributed data that decouples ML models from cryptographic protocols, efficiently supporting horizontal and vertical FL across multi-party, multi-host clusters with secure multi-party computation (MPC) primitives. Allouah et al. (2024) propose Decor, a decentralized SGD variant with differential privacy guarantees using pairwise-canceling correlated Gaussian noise – introducing SecLDP (secret-based local differential privacy), which protects user communications against external eavesdroppers and curious users under the assumption that connected user pairs share secrets.

Bienstock et al. (2025) introduce DMM (Distributed Matrix Mechanism) for differentially-private federated learning, achieving both distributed DP and the privacy-utility trade-off of central DP via a constant-overhead linear secret resharing protocol that securely transfers values across client committees with dynamic participation. Elangovan et al. (2024) present communication-efficient secure and private multi-party deep learning, combining differential privacy with MPC protocols – achieving $56\text{-}794\times$ communication efficiency and $16\text{-}182\times$ speedup over prior DP-MPC approaches. Sabater et al. (2025) propose CAFCOR, a privacy-preserving and robust distributed learning algorithm under SecLDP, addressing untrusted servers and malicious workers through correlated noise injection and a novel robust aggregation method – achieving near-CDP performance under limited corruption.

2.6 Non-IID Data Challenges in Federated Learning

Non-IID data remains the central unsolved challenge in federated learning, degrading model performance and slowing convergence when client data distributions diverge. Jiménez et al. (2024) survey non-IID data in federated learning, providing a taxonomy for data heterogeneity (label skew, feature distribution skew, quantity skew, and temporal/spatial skew), partition protocols, and metrics to quantify heterogeneity. The survey concludes no universal solution exists; methods must be tailored to specific heterogeneity types.

Gu et al. (2024) propose HFLDD (Hybrid Federated Learning with Dataset Distillation), partitioning clients into heterogeneous clusters with balanced inter-cluster label distributions and unbalanced intra-cluster distributions; cluster heads collect distilled (approximately IID) data from members, enabling training similar to traditional FL on IID data – effectively alleviating non-IID impact. Xu et al. (2024) propose FedPRL, a personalized federated learning approach using reinforcement learning for client selection and global knowledge distillation of non-target classes, addressing catastrophic forgetting during global updates – enhancing generalization and personalized model performance under data and system heterogeneity.

Zhang et al. (2025) introduce Fed-GDBD (Federated learning with Global Decision Boundary Distillation), supporting heterogeneous models across clients by distilling global decision boundaries rather than aggregating gradients – addressing both data and model heterogeneity simultaneously. Bai et al. (2025) propose Fed-Hydra, a unified solution to diverse heterogeneities in one-shot federated learning (single communication round), using auxiliary public datasets and embedding techniques to mitigate data, model, and system heterogeneity without multi-round communication.

2.7 Class Imbalance and Ensemble Methods

2.7.1 SMOTE and Class Imbalance

Synthetic Minority Over-sampling Technique (SMOTE) is widely deployed to handle class imbalance in IoT datasets. The baseline paper Et-Tousy et al. (2026) applies SMOTE to balance its 48%/42%/10% three-class QoS decision problem. Recent work confirms SMOTE’s effectiveness across IoT domains.

Haider et al. Haider et al. (2023) evaluate SMOTE with KNN, SVM, and ANN on the IoT-23 dataset, showing SMOTE improves accuracy across all three algorithms, albeit differently per algorithm. Kharisma et al. Kharisma et al. (2024) evaluate sampling methods (SMOTE, ADASYN, undersampling) with five classifiers on IoTID20; SMOTE-XGBoost achieves best performance (75% accuracy, 82% weighted precision, 78% F1-score). Ibrahim et al. Ibrahim et al. (2024) study BoT-IoT balancing with SMOTE, reporting minority class F1-score increases from 0.77 to 1.0 (KNN), 0 to 0.989 (Gradient Boosting), and 0 to 0.999 (SVM) after balancing. Da Silva et al. da Silva et al. (2025) apply SMOTE to the CICIoT2023 dataset with lightweight classifiers (Bernoulli NB, Decision Tree, Random Forest, XGBoost); XGBoost and Random Forest with SMOTE achieve high, balanced detection across 34 attack types.

Saini et al. Saini et al. (2025) combine SMOTE with Transformers and black-winged kite algorithm-based feature selection for IoT intrusion detection on BoT-IoT, TON-IoT, and CIC-DDoS2019, addressing imbalance as a first-class concern.

2.7.2 Ensemble Methods for Edge/IoT

Random Forests and ensemble methods are favored for edge deployment due to their interpretability, robustness, and parallelizability. Alkhoury et al. Alkhoury et al. (2025) introduce Splitting Stump Forests (SSF), compressing trained Random Forests into small ensembles of weak learners by selecting nodes that evenly split training data and applying linear models on the representation – achieving two-to-three orders of magnitude model-size reduction with minimal accuracy loss, suitable for memory-limited embedded devices including Arduino deployments. Verosimile et al. Verosimile et al. (2025) propose Moyogi, a memory-centric hardware-software codesign accelerator for Random Forest inference on embedded IoT devices, achieving $3.88\times$ geomean latency reduction on IoT datasets. Haraguchi et al. Haraguchi et al. (2025) survey decision-tree-based distributed learning for IoT, highlighting Random Forest and Gradient Boosting Decision Trees (GBDT) as lightweight, distributed alternatives to deep neural networks for resource-constrained devices.

2.8 Autonomic Computing with MAPE-K

The MAPE-K (Monitor, Analyze, Plan, Execute, Knowledge) loop, introduced by Kephart and Chess Kephart and Chess (2003) as a reference architecture for autonomic computing, provides a structured framework for self-adaptive systems. The baseline paper Et-Tousy et al. (2026) employs MAPE-K to integrate ML-driven QoS decisions into a continuous monitoring and adaptation loop.

Recent work extends MAPE-K to IoT and edge contexts. Riegler et al. Riegler et al. (2023) propose DSec4IoT, a distributed MAPE-K framework for self-protective IoT devices, leveraging distributed MAPE-K patterns to enable resource-constrained devices

to detect and mitigate security threats (e.g., port scans) without relying solely on centralized infrastructure. Stadler et al. (2024) apply multi-level MAPE-K loops to cyber-resilient edge computing, demonstrating effectiveness on real-world industrial edge devices. Alfonso et al. (2023) present a model-based MAPE-K framework for IoT systems in wastewater treatment plants, combining domain-specific language (DSL) modeling with MAPE-K-driven runtime adaptation to manage sensor data and process block diagrams.

2.9 Summary: Positioning This Work

The literature establishes federated learning as a mature approach for privacy-preserving distributed ML across IoT and edge systems, confirms that non-IID data remains the central challenge, validates SMOTE and ensemble methods (especially Random Forest) as effective for imbalanced IoT classification tasks, and demonstrates MAPE-K as a proven framework for autonomic QoS management. However, no prior work combines these elements as the baseline paper Et-Tousy et al. (2026) proposes: a federated Random Forest alternative to centralized QoS-driven offload decisions in OneM2M, evaluated explicitly against the centralized baseline and a local-only lower bound on non-IID synthetic multi-site telemetry.

This project fills exactly the gap the baseline paper names: it reproduces the centralized Random Forest classifier on multi-site QoS telemetry, builds the federated alternative the paper proposes as future work, and quantifies the privacy-utility trade-off by measuring what each site loses by going it alone versus federating versus full centralization. The federated ensemble design (per-site local Random Forests combined via probability averaging) is simpler than weighted aggregation schemes surveyed in G. et al. (2024) and Abreha et al. (2026), and does not employ secure aggregation as in Bienstock et al. (2025) or Allouah et al. (2024) – these are explicitly identified as future work in Section 7.

3 Methodology

This section describes the experimental methodology used to reproduce the baseline paper’s centralized Random Forest classifier and to build and evaluate the federated alternative. The methodology is designed to answer the research question: can a federated learning approach recover most of the centralized model’s accuracy while addressing the privacy and scalability limitations the baseline paper identifies?

3.1 Overview

Figure ?? illustrates the multi-site IoT QoS architecture. Six simulated IoT edge sites generate local QoS telemetry with non-IID traffic distributions. Figure ?? contrasts the centralized training approach (pooling all sites’ raw data) with the federated approach (training locally per site, combining predictions only).

3.2 Synthetic Multi-Site QoS Telemetry

Real multi-site OneM2M telemetry was not available, so a synthetic generator (`src/data/qos_simulator`) produces QoS records for 6 simulated IoT edge sites. Each record contains six features drawn from the baseline paper’s own QoS telemetry schema:

- **RTT (Round-Trip Time):** milliseconds, primary traffic-oriented metric
- **CPU utilization:** percentage (0-100%)
- **RAM utilization:** percentage (0-100%)
- **Success rate:** percentage (0-100%), proportion of successful requests
- **Latency:** milliseconds, application-layer delay
- **Throughput:** requests/second, system capacity

Each site’s traffic is a mixture of the baseline paper’s own three scenarios – *uniform* (steady-state traffic), *burst* (periodic load spikes), and *real-time* (time-sensitive requests with tight latency bounds) – but with a different, skewed mixture per site: 80% dominated by one scenario, 10% each of the other two. This skew is what makes this a genuine non-IID federated learning setting: no single site sees a representative sample of the overall traffic distribution. 1,500 records per site are generated per random seed, for a total of 9,000 records per seed across all six sites.

3.2.1 Decision Labels and Thresholds

Each record is labeled using the baseline paper’s exact thresholds:

- **Class 0 (Keep Local / Preferable):** $\text{RTT} < 2\text{s}$, $\text{success rate} > 95\%$, $\text{CPU} < 70\%$, $\text{RAM} < 70\%$
- **Class 1 (Partial Offload / Acceptable):** RTT between 2s and 4s, success rate 90-95%, CPU 70-80%, RAM 70-80%
- **Class 2 (Full Offload / Critical):** $\text{RTT} > 4\text{s}$, $\text{success rate} < 90\%$, $\text{CPU} > 80\%$, or $\text{RAM} > 80\%$

The synthetic generator applies these thresholds deterministically to produce class labels, then adds small amounts of Gaussian noise to features to simulate measurement error without changing labels. Class imbalance matches the baseline paper’s reported distribution: approximately 48% Class 0, 42% Class 1, 10% Class 2 (averaged across all sites).

3.3 Training Pipeline

For each of 5 random seeds (42, 43, 44, 45, 46), the following procedure is executed:

1. **Data split:** Each site’s 1,500 records are split 80/20 (stratified by label) into a local training set (1,200 records) and a local test set (300 records). The local test sets from all six sites are pooled into a single held-out test set (1,800 records total) used to evaluate all models.
2. **Centralized training (baseline reproduction):** All sites’ training sets ($6 \times 1,200 = 7,200$ records) are pooled into one centralized training set. SMOTE (Synthetic Minority Over-sampling Technique) is applied to balance the three classes to equal proportions. A single Random Forest classifier (200 trees, max depth 10, Gini impurity splitting criterion) is trained on the SMOTE-balanced pooled data.

3. **Federated training (proposed improvement)**: For each of the 6 sites independently:

- Apply SMOTE locally to that site’s training set (1,200 records) to balance the three classes within that site.
- Train a Random Forest classifier (200 trees, max depth 10, Gini splitting) on the locally SMOTE-balanced data.
- Store the trained model; no data or model parameters are shared with any other site or central server at this stage.

At inference time, each of the 6 site models predicts class probabilities for a given test instance; these 6 probability vectors are averaged element-wise, and the class with the highest averaged probability is returned as the ensemble prediction.

4. **Single-site local-only training (lower bound)**: For each of the 6 sites independently, the same locally SMOTE-balanced Random Forest (200 trees, max depth 10) is trained as in the federated case, but evaluated in isolation on the pooled test set with no combination with any other site’s model. This represents the performance a site achieves if it can neither centralize data nor federate with other sites.

All three variants (centralized, federated, single-site local-only) use identical Random Forest hyperparameters. The only difference is the data-sharing strategy: centralized pools raw data; federated trains locally and combines predictions; single-site uses only its own local model. This design isolates the effect of the data-sharing strategy on model performance.

3.4 Evaluation Metrics

All models are evaluated on the same pooled, held-out test set (20% of every site’s data, stratified by label, never used in training by any model). Three metrics are reported:

- **Accuracy**: overall proportion of correct predictions across all three classes.
- **Macro-F1 score**: unweighted average of per-class F1 scores, giving equal importance to all three classes regardless of their frequency in the test set. This metric is preferred over weighted-F1 or micro-F1 because it does not let the majority classes dominate the evaluation.
- **Critical Recall (class 2 recall)**: recall specifically on the “Full Offload” (Critical) class. This is the minority class the baseline paper needed SMOTE to handle (9.64% of their real data), and the one most consequential to miss in production: a false negative on Critical means an overloaded system incorrectly stays local, risking degraded user experience or system failure.

Results are reported as mean $\pm$ standard deviation across 5 seeds. Statistical significance of differences between variants is not formally tested due to the small number of seeds, but consistency across seeds is noted qualitatively.

3.5 Implementation Details

All models are implemented in Python 3 using scikit-learn 1.3 for Random Forest and imbalanced-learn 0.11 for SMOTE. Training and evaluation are performed on a single machine (Intel Xeon CPU, 32GB RAM, no GPU required). Total runtime for all three variants across 5 seeds is approximately 15 minutes. Trained models are serialized via joblib for deployment; the federated ensemble’s 6 per-site models total 42MB on disk (7MB per model), comparable to the centralized model’s single 8MB file.

All code, synthetic telemetry generator, training scripts, and results are available in `iot-reliability/`.

4 Design Specification

This section provides the detailed design specification for the three model variants evaluated in this project: the centralized Random Forest baseline, the federated ensemble alternative, and the single-site local-only lower bound.

4.1 Design Goals

The design is driven by three goals:

1. **Faithful baseline reproduction:** Reproduce the centralized Random Forest classifier from Et-Tousy et al. Et-Tousy et al. (2026) as closely as possible given synthetic data, using their exact decision thresholds, SMOTE-based class balancing, and Random Forest hyperparameters.
2. **Privacy-preserving federation:** Build a federated alternative where no site’s raw QoS telemetry ever leaves that site – only trained model predictions are shared at inference time, addressing the baseline paper’s named privacy and centralization concerns.
3. **Quantifiable trade-off:** Measure the privacy-utility trade-off by comparing three points: (a) full centralization (best utility, worst privacy), (b) federation (middle ground), and (c) local-only (worst utility, best privacy for a single site). This three-point comparison establishes whether federation is actually worth building relative to going it alone.

4.2 Decision Labels and Thresholds

Each QoS record is labelled using the baseline paper’s own thresholds:

- **Class 0: Keep Local (Preferable):** System is operating within normal parameters. RTT < 2s, success rate > 95%, CPU utilization < 70%, RAM utilization < 70%. All processing remains on the local IoT edge server.
- **Class 1: Partial Offload (Acceptable):** System is experiencing moderate load. RTT between 2s and 4s, success rate 90-95%, CPU 70-80%, RAM 70-80%. Some non-critical tasks can be offloaded to the cloud while latency-sensitive processing stays local.

- **Class 2: Full Offload (Critical):** System is overloaded or approaching failure. $RTT > 4s$, success rate $< 90\%$, CPU utilization $> 80\%$, or RAM utilization $> 80\%$. All possible workload should be migrated to cloud resources to prevent local system degradation or failure.

These thresholds match the baseline paper’s MAPE-K monitoring loop’s state classification exactly. In production deployment, the model’s predicted class drives the “Plan” stage of the MAPE-K loop, triggering automatic traffic redistribution actions.

4.3 Centralized RF (baseline reproduction)

The centralized variant reproduces the baseline paper’s approach:

1. **Data pooling:** All sites’ raw QoS telemetry (RTT, CPU, RAM, success rate, latency, throughput) is pooled into one centralized training set. For the synthetic dataset, this is 7,200 training records ($6 \text{ sites} \times 1,200 \text{ records per site}$).
2. **Class balancing:** SMOTE (Synthetic Minority Over-sampling Technique) is applied to the pooled training set to balance the three classes to equal proportions. The baseline paper reports a 48%/42%/10% class distribution in their real Azure IoT data; our synthetic generator produces a similar distribution (approximately 48%/42%/10% averaged across sites, with site-specific variation due to non-IID skew). SMOTE up-samples the minority class (Class 2) by generating synthetic instances along line segments connecting existing minority-class samples to their k-nearest neighbors ($k=5$ by default).
3. **Model training:** A single Random Forest classifier is trained on the SMOTE-balanced pooled data. Hyperparameters are set to match the baseline paper’s best-performing model: 200 trees, maximum depth 10, Gini impurity splitting criterion, minimum samples per leaf 1, minimum samples per split 2. No pruning or feature selection is applied.
4. **Deployment:** The trained centralized model is evaluated on the pooled held-out test set. In a production deployment, this model would be hosted on a central server or cloud platform, receiving QoS telemetry from all sites in real time and returning offload decisions.

Privacy limitation: This design requires all sites to transmit their raw QoS telemetry to a central location for training and potentially for inference (if the model is centrally hosted). This is the privacy/centralization concern the baseline paper names in its Discussion section.

4.4 Federated Ensemble RF (proposed improvement)

The federated variant addresses the baseline’s privacy limitation:

1. **Local training:** Each of the 6 simulated IoT sites trains its own Random Forest classifier independently, using the same hyperparameters as the centralized model (200 trees, max depth 10). Training data is the site’s own local telemetry only (1,200 records per site), never transmitted to any other site or central server.

2. **Local class balancing:** SMOTE is applied locally within each site to balance the three classes within that site’s training set. Because each site has a skewed traffic distribution (80% one scenario, 10% each of the other two), SMOTE must work with less diverse minority-class samples than the centralized case. This is a key disadvantage of the federated approach: local SMOTE cannot generate synthetic samples that bridge the gap between different sites’ distributions.
3. **Model storage:** Each site stores its trained Random Forest model locally. Models are not transmitted to any other site or central coordinator. No model parameters or gradients are shared – the models are entirely independent.
4. **Inference-time combination:** At inference time, when a new QoS record must be classified:
 - Each of the 6 site models predicts a class probability distribution over the 3 classes: $\mathbf{p}_i = [p_{i,0}, p_{i,1}, p_{i,2}]$ for site i .
 - The 6 probability vectors are averaged element-wise: $\mathbf{p}_{\text{ensemble}} = \frac{1}{6} \sum_{i=1}^6 \mathbf{p}_i$.
 - The class with the highest averaged probability is returned: $\arg \max_c p_{\text{ensemble},c}$.

This simple probability-averaging scheme requires no training or tuning beyond the per-site Random Forests. More sophisticated aggregation schemes (e.g., weighting each site by its local validation performance, or learning a meta-model over the site predictions) are left as future work.

5. **Deployment:** In a production deployment, the 6 site models could be hosted on their respective local edge servers, or replicated to a central inference server that applies the averaging. Crucially, only the trained models (and their predictions on specific instances) are ever transmitted; raw telemetry never leaves the site that generated it.

Privacy guarantee: No site’s raw QoS telemetry is ever transmitted to or pooled with another site’s data. Training is entirely local. Only model predictions (probabilities over 3 classes) are combined at inference time. This addresses the centralization concern the baseline paper names.

Limitation: The federated ensemble does not employ secure aggregation or differential privacy, so a malicious central inference server could inspect individual site models’ predictions and potentially infer properties of the training data. Secure aggregation and differential privacy are explicitly identified as future work in Section 7.

4.5 Single-Site Local Only (lower bound)

To establish whether federating is actually worth the engineering effort, each site’s own local model is also evaluated in isolation:

1. **Training:** Identical to the federated case – each site trains a Random Forest (200 trees, max depth 10) on its own locally SMOTE-balanced training set (1,200 records).

2. **Evaluation:** Each site’s model is evaluated individually on the pooled held-out test set (1,800 records from all 6 sites), with no combination with any other site’s model. This measures the performance a single site achieves if it can neither share data (centralization) nor collaborate on a model (federation).
3. **Reporting:** Results are averaged across all 6 sites to produce a mean $\pm$ standard deviation. This represents the expected performance of a site chosen uniformly at random from the population, under the assumption that it cannot leverage any other site’s data or model.

This variant establishes the lower bound on performance: what a site is stuck with if it must go it alone. Comparing single-site to federated quantifies the benefit of collaboration without centralization; comparing federated to centralized quantifies the cost of preserving privacy.

5 Implementation

The system is implemented in Python 3 with scikit-learn and imbalanced-learn. Key modules:

- `src/data/qos_simulator.py` – synthetic multi-site QoS telemetry generator described in Section 3.
- `src/models/qos_models.py` – `CentralizedRF` (baseline) and `FederatedEnsembleRF` (improvement), both wrapping scikit-learn’s `RandomForestClassifier` with SMOTE balancing via imbalanced-learn.
- `scripts/train_and_evaluate.py` – trains and evaluates all 3 variants across 5 seeds, writes `results/results_per_seed.csv` and `results/results_summary.csv`, and exports one trained model per site (via joblib) to `src/lambda_handler/model/` for deployment.
- `src/lambda_handler/app.py` – an AWS Lambda handler that loads all 6 exported per-site models and applies the federated ensemble decision to each incoming QoS telemetry record arriving on a Kinesis stream, returning a Keep-Local/Partial-Offload/Full-Offload decision with class probabilities.
- `template.yaml` – AWS SAM template provisioning the Kinesis stream and Lambda function.
- `.github/workflows/deploy.yml` (repo root) – CI pipeline that runs the training/evaluation script and, on push to `main`, deploys the SAM stack to the configured AWS region using repository secrets.

Running `python scripts/train_and_evaluate.py` reproduces every number in Section 6 from scratch. The Lambda handler was verified locally against representative telemetry for all three decision classes before deployment.

Table 1: Results across 5 seeds (mean $\pm$ std)

Model	Accuracy	Macro-F1	Critical Recall (class 2)
Centralized RF (baseline)	0.9995 $\pm$ 0.0005	0.9996 $\pm$ 0.0005	0.9994 $\pm$ 0.0013
Federated Ensemble RF (improved)	0.9959 $\pm$ 0.0024	0.9949 $\pm$ 0.0024	0.9997 $\pm$ 0.0007
Single-Site Local Only (no federation)	0.9761 $\pm$ 0.0046	0.9737 $\pm$ 0.0051	0.9958 $\pm$ 0.0031

6 Evaluation

Table 1 reports mean $\pm$ standard deviation across 5 seeds (42-46) on a pooled, held-out test set drawn from all 6 sites.

6.1 Experiment 1: The Cost of Going It Alone

A single site’s own local model, used in isolation with no federation and no centralization, reaches only 97.4% macro-F1 – a clear, consistent gap below both the centralized (99.96%) and federated (99.49%) variants. This confirms the premise of the whole exercise: because each site’s local traffic mixture is skewed toward one of the three scenarios (uniform/burst/real-time), a site that only ever sees its own data generalises worse to the full population it will actually encounter in production.

6.2 Experiment 2: Federation Recovers Most of the Centralization Benefit

The federated ensemble closes the large majority of the gap between local-only and centralized (macro-F1 97.37% $\rightarrow$ 99.49%, versus centralized’s 99.96%), while never pooling raw telemetry across sites – directly addressing the privacy/centralization limitation the baseline paper names in its own Discussion section. The remaining gap to full centralization (99.49% vs. 99.96% macro-F1, about half a point) is the measurable cost of keeping data decentralized, and is small relative to the 2.6-point gap federation recovers from the single-site baseline.

6.3 Experiment 3: The Minority Class

Critical Recall (the class the baseline paper itself needed SMOTE to handle, at only 9.64% of their real-world data) is high and statistically indistinguishable across all three variants (99.6-100%), including the single-site baseline. This suggests SMOTE-balancing within each site is sufficient on its own to protect the minority class here; federation’s main benefit in this experiment is on overall accuracy/macro-F1 (driven by better generalisation across traffic regimes), not on minority-class recall specifically.

6.4 Discussion

All three variants score unusually high in absolute terms (97-100%) compared to the baseline paper’s own reported 96-98% for its best models. This is expected: the synthetic generator’s labelling rule is a deterministic function of the same features the models are trained on, with less real-world sensor noise than Azure IoT telemetry would have. The relative ordering of the three variants – local-only worst, federated in the middle,

centralized best – is the finding that matters here, not the absolute numbers, and that ordering is what a privacy/utility trade-off in federated learning would predict.

7 Conclusion and Future Work

This project asked whether a federated learning alternative to Et-Tousy et al.’s (2026) centralized Random Forest – one model per IoT site trained on local data only, combined without pooling raw telemetry – could recover most of the centralized model’s accuracy while resolving the privacy/scalability limitation the paper names but does not build. Across 5 seeds on synthetic, non-IID multi-site QoS telemetry, the federated ensemble reached 99.49% macro-F1 versus the centralized baseline’s 99.96% and a single site’s own local-only model’s 97.37% – recovering the large majority of the benefit of centralization without requiring it.

The clearest finding is not the small centralized-vs-federated gap, but the large local-only-vs-federated gap: a site that cannot collaborate with any other site (no data sharing, no federation) generalises measurably worse to the traffic patterns it will actually see in production. Federation is the difference between a site being stuck with a locally-biased model and getting most of the benefit of the full population’s data, without that data ever leaving its site of origin.

Future work:

- Evaluate on real multi-site OneM2M/Azure IoT telemetry rather than synthetic data, to check whether the federation benefit holds under real sensor noise and real non-IID splits.
- Extend the federated design from simple probability-averaging to a weighted aggregation scheme (e.g. weighting each site’s vote by its local validation confidence), which the current implementation does not attempt.
- Evaluate secure aggregation (so that not even model updates are individually inspectable by a central coordinator), extending the privacy property this project targets but does not fully secure.

References

- Abreha, H. D. et al. (2026). Federated learning for edge computing enabled artificial intelligence of things: A comprehensive survey, *Knowledge-Based Systems* **312**: 116300.
- Alfonso, I., Rodríguez, A., Pérez-Vereda, A., Canal, C. and Pimentel, E. (2023). A model-based framework for IoT systems in wastewater treatment plants, *Journal of Reliable Intelligent Environments* **22**(2): a10.
- Alkhoury, F., Buschjäger, S. and Welke, P. (2025). Splitting stump forests: tree ensemble compression for edge devices (extended version), *Machine Learning* **114**: 219.
- Allouah, Y., Guerraoui, R., Gupta, N., Stoica, A.-A. and Stephan, J. (2024). The privacy power of correlated noise in decentralized learning, *Proceedings of the 41st International Conference on Machine Learning (ICML)*, Vol. 235 of *PMLR*, pp. 1115–1143.

- B, G. E. et al. (2024). Efficient OneM2M standard implementation for lightweight IoT, *2024 8th International Conference on Devices, Circuits and Systems (DSIT)*, pp. 1–6.
- Badrinarayanan, S., Das, S., Garimella, G., Raghuraman, S., Rindal, P. and Shah, A. (2024). Communication efficient secure and private multi-party deep learning, IACR ePrint Archive, Report 2024/1471. <https://eprint.iacr.org/2024/1471>.
- Bai, J., Song, Y., Wu, D., Sajjanhar, A., Xiang, Y., Zhou, W., Tao, X., Li, Y. and Li, Y. (2025). A unified solution to diverse heterogeneities in one-shot federated learning, *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, pp. 1–12.
- Bienstock, A., Das, S., Joshi, P., Baldi, M., Balle, B., Kairouz, P., Ramage, D. and Kumar, R. (2025). DMM: Distributed matrix mechanism for differentially-private federated learning based on constant-overhead linear secret resharing, *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, Vol. 267 of PMLR.
- da Silva, V. F., Gonçalves, V. P. and Couto, R. S. (2025). Synthetic minority over-sampling technique for detecting malicious traffic targeting internet of things’ devices, *Journal of Internet Services and Applications* **16**(1): 1–17.
- Et-Tousy, J. and Zyane, A. (2025). Automated QoS enhancement in OneM2M: Traffic-oriented approach for MQTT and HTTP based on IoT SLA variations, *Lecture Notes in Networks and Systems* **1306**: 126–139.
- Et-Tousy, J., Zyane, A. and Sharif, S. (2025). Machine learning-driven QoS optimization for IoT in OneM2M: A novel approach for traffic and resource management, *2025 11th International Conference on Future Internet of Things and Cloud (FiCloud)*, pp. 134–141.
- Et-Tousy, J., Zyane, A. and Sharif, S. (2026). Adaptive QoS management in OneM2M standard: Machine learning and deep learning for IoT network optimization, *Journal of Network and Systems Management*. Preprint: <https://doi.org/10.21203/rs.3.rs-7987618/v1>.
- G., D. M. J., Solans, D., Heikkilä, M., Vitaletti, A., Kourtellis, N., Anagnostopoulos, A. and Chatzigiannakis, I. (2024). Non-IID data in federated learning: A survey with taxonomy, metrics, methods, frameworks and future directions, arXiv preprint arXiv:2411.12377.
- Gu, Y., Li, P., Wang, S. and Deng, Y. (2024). Dataset distillation-based hybrid federated learning on non-IID data, arXiv preprint arXiv:2409.17517.
- Haider, W., Butt, M. H. F., Yasmin, S. and Yasin, A. (2023). Machine learning techniques evaluation with SMOTE on IoT-23 dataset, *2023 International Conference on Emerging Power Technologies (ICEPT)*, pp. 1–6.
- Haraguchi, T., Tsukamoto, Y., Soshi, M. and Koike, Y. (2025). Decision-tree-based distributed learning for IoT devices, *Journal of Grid Computing* **23**: 14.

- Ibrahim, M., Torky, M., El-Moursy, A., Ali, H. A. and Abdelhamid, A. A. (2024). STUDYING THE IMPACT OF DATASET BALANCING ON MACHINE LEARNING-BASED INTRUSION DETECTION SYSTEMS FOR IOT, *International Journal of Intelligent Computing and Information Sciences* **24**(4): 27–42.
- Ihita, G. V., Acharya, V. K., Kanigolla, L., Chaudhari, S. and Monteil, T. (2023). Security for oneM2M-based smart city network: An OM2M implementation, *2023 15th International Conference on COMMunication Systems & NETWORKS (COMSNETS)*, pp. 808–813.
- Kephart, J. O. and Chess, D. M. (2003). The vision of autonomic computing, *Computer* **36**(1): 41–50.
- Kharisma, A. E., Prayudi, Y. and Riadi, I. (2024). Sampling-based machine learning models for intrusion detection in imbalanced dataset, *Electronics* **13**(10): 1878.
- Kim, D., Jo, U., Kim, Y., Oktian, Y. E. and Kim, H. (2023). Design and implementation of a blockchain based interworking of oneM2M and LWM2M IoT systems, *Journal of Information Processing Systems* **19**(1): 89–97.
- Li, Y. (2024). Survey: federated learning data security and privacy-preserving in edge-Internet of Things, *Artificial Intelligence Review* **57**: 248.
- Li, Y., Liu, Y., Yuan, L., Fan, Q. et al. (2024). Mobility-aware partial task offloading and resource allocation based on deep reinforcement learning for mobile edge computing, *IEEE Transactions on Vehicular Technology*.
- Minwuye, T. et al. (2025). Federated learning in IoT environments: Examining the three-way see-saw for privacy, model-performance, and network-efficiency, *IEEE Communications Surveys & Tutorials*.
- Olvera-Córdova, P., González-Briones, S., González-Briones, A., Corchado, J. M. and Chamoso, P. (2024). Deep reinforcement learning techniques for dynamic task offloading in the 5g edge-cloud continuum, *Journal of Cloud Computing* **13**(94).
- Pang, Z., Zhang, X., Wang, C., Wang, X. and Xu, L. (2025). DAPO: Mobility-aware joint optimization of model partitioning and task offloading for edge LLM inference, *Electronics* **14**(19): 3929.
- Qu, J., Ullah, A., Wang, P., Ali, M. and Wang, F. (2025). KroQoSNet: A lightweight QoS prediction framework for adaptive protocol selection in IoT, *2025 IEEE International Conference on IoT and AAI*.
- Riegler, M., Sametinger, J. and Vierhauser, M. (2023). A distributed MAPE-K framework for self-protective IoT devices, *2023 IEEE 18th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS)*, pp. 202–208.
- Sabater, C., Farhadkhani, S., Guerraoui, R. and Allouah, Y. (2025). Towards trustworthy federated learning with untrusted participants, arXiv preprint arXiv:2505.01874.

- Saini, U., Jain, S. and Rattan, D. (2025). Overcoming dataset imbalances and computational challenges in IoT intrusion detection: A SMOTE-enhanced transformer-based model, *2025 15th International Conference on Cloud System and Big Data Engineering (CSNT)*, pp. 1–6.
- Stadler, M., Riegler, M. and Sametinger, J. (2024). Cyber-resilient edge computing: A holistic approach with multi-level MAPE-K loops, *2024 IEEE 21st International Conference on Software Architecture Companion (ICSA-C)*, pp. 79–83.
- Trigka, M. and Dritsas, G. (2025). Federated learning for IoT: A survey of techniques, challenges, and applications, *Journal of Sensor and Actuator Networks* **14**(1): 9.
- Verosimile, A., Peverelli, F. and Santambrogio, M. D. (2025). Moyogi: A memory-centric accelerator for low-latency random forest inference on embedded devices, *2025 IEEE 33rd International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, pp. 1–9.
- Vo, N. K., Thanh, N. H., Luong, D. C. and Tran, X. T. (2023). Enhanced interop-erating mechanism between OneM2M and OCF platform based on rules engine and interworking proxy in heterogeneous IoT networks, *IEEE Access* **11**: 6839–6857.
- Wu, J., Dong, F., Leung, H., Zhu, Z., Zhou, J. and Drew, S. (2024). Topology-aware fed-erated learning in edge computing: A comprehensive survey, *ACM Computing Surveys* **56**(10): 1–41.
- Xiao, H., Hu, Z., Zhang, X., Xu, A., Zheng, M. and Li, K. (2024). Federated deep rein-forcement learning for task offloading in mobile edge computing-enabled heterogeneous networks, *IEEE Internet of Things Journal* .
- Xu, K., Chai, D., Zhang, J., Lai, F. and Chen, K. (2025). Sequoia: An accessible and extensible framework for privacy-preserving machine learning over distributed data, *Proceedings of the ACM on Management of Data* **3**(1): 74:1–74:27.
- Xu, L., Mei, S., Liu, H., Hou, Y. and Li, L. (2024). An efficient personalized federated learning approach in heterogeneous environments: a reinforcement learning perspec-tive, *Scientific Reports* **14**: 29486.
- Zhang, K., Wang, J., Wang, W., Zhou, X., Tan, Z. and Wang, Z. (2025). Federated learn-ing with heterogeneous data and models based on global decision boundary distillation, *Journal of King Saud University - Computer and Information Sciences* **37**(1): 75.
- Zhang, Q., Zhao, L., Chen, Z. and Chen, Q. (2024). Hybrid deep reinforcement learning-based task offloading for D2D-assisted cloud-edge-device collaborative networks, *IEEE Transactions on Mobile Computing* .
- Zyane, A. and Et-Tousy, J. (2025). Integrating deep learning models for QoS optimization in IoT using the OneM2M platform, *2025 11th International Conference on Future Internet of Things and Cloud (FiCloud)*, pp. 94–101.